Gyűjtemények, felsorolók, algoritmus minták

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Gyűjtemény és feldolgozása

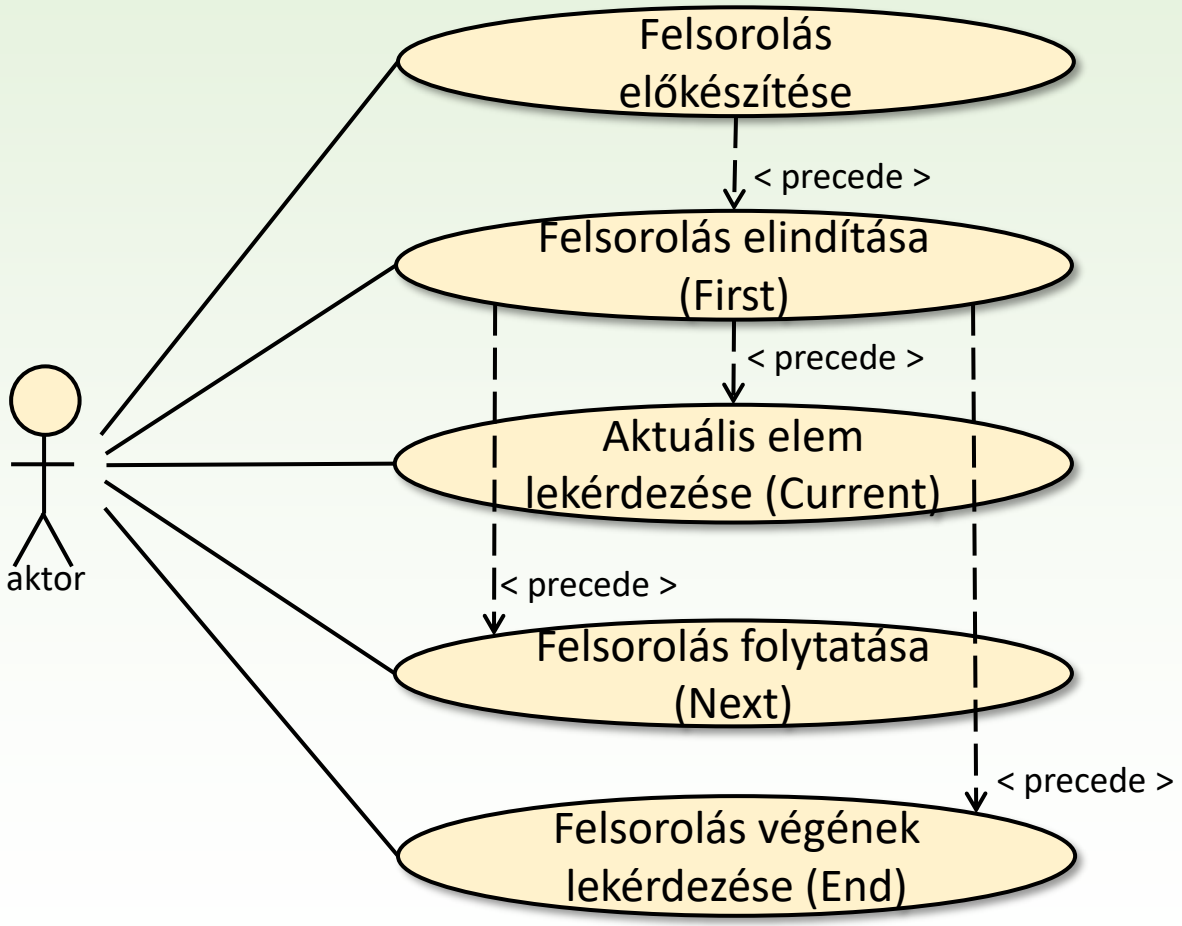
- A gyűjtemény (tároló, kollekció) egy olyan objektum, amely elemek tárolására alkalmas, és az eltároláshoz, valamint a visszakereséshez biztosít műveleteket. Ilyenek például az
  - iterált szerkezetű (azonos típusú elemeket tartalmazó) objektumok
    - Pl.: halmaz, zsák, sorozat (verem, sor), tömb, fa, gráf
- Egy gyűjtemény lehet virtuális is, amelyiknek nem tároljuk explicit módon az elemeit
    - Pl.: egész számok egy intervalluma, természetes szám prím-osztói
- Egy gyűjtemény feldolgozásán a benne levő elemek feldolgozását értjük.
  - Keressük egy halmaz valamilyen szempont szerinti legnagyobb elemét!
  - Hány negatív szám van egy számsorozatban?
  - Keressük meg egy egészeket tartalmazó tömb azon pozitív elemét, amelyet a tömb visszafelé bejárásával elsőként kapunk meg úgy, hogy csak minden második elemet vizsgáljuk meg!
  - Soroljuk fel az *n* természetes szám pozitív prím-osztóit!

Felsorolás

- Egy gyűjtemény feldolgozásához szükségünk van a gyűjtemény elemeinek felsorolására (bejárására).
- Erre a felsorolásra úgy tekinthetünk, mint a gyűjtemény (E halmazhoz tartozó) elemeiből képzett véges sorozatra, amelyre az alábbi műveleteket használhatjuk:
  - First() : rááll a sorozat első elemére, azaz elkezdi a felsorolást
  - Next() : rááll a sorozat soron következő elemére, azaz folytatja a felsorolást
  - l := End() (l:𝕃) : megmutatja, hogy a sorozat, azaz a felsorolás végére értünk-e
  - e:= Current() (e:E): visszaadja a sorozat, a felsorolás aktuális elemét

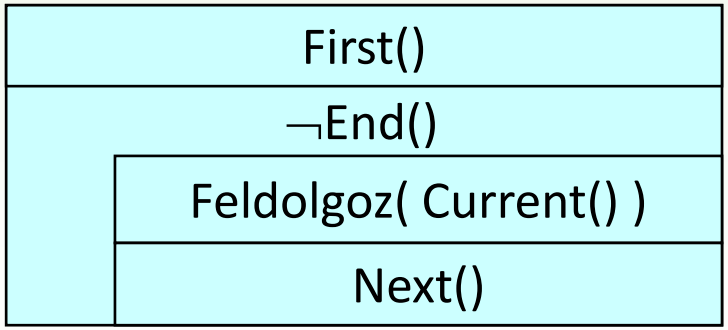
Használati eset diagram

- A felsorolásának használati eseteinek meghatározott sorrendje van.



Felsorolás állapotai

- Ahhoz, hogy a felsorolás tevékenységeinek rákövetkezési feltételeit betartsuk, érdemes megkülönböztetni egy felsorolás különböző állapotait: *indulásra kész*, *folyamatban van*, *befejeződött*.
- A felsorolás műveletei csak bizonyos állapotokban értelmesek, máskor a hatásuk nem definiált.
- Célszerű a felsorolást olyan algoritmusba beágyazni, amely garantálja, hogy a felsoroló műveletei csak akkor (olyan állapotban) kerüljenek végrehajtásra, amikor az értelmes.




```
for( first(); !end(); next() )
{
        process(current());
}
```

a felsorolt elemek típusát helyettesíti

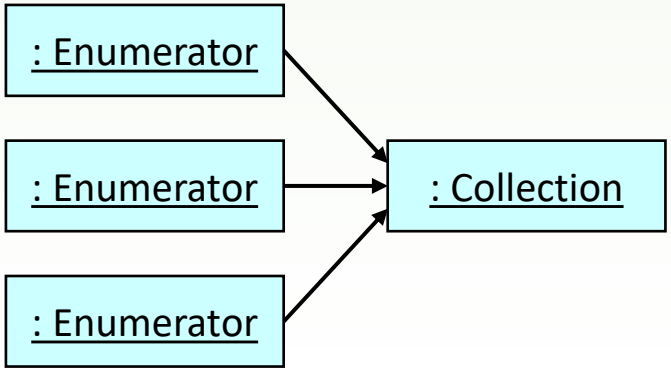
foreach (forall) ciklus: gyűjtemény felsorolása

```
for( auto e : h )
{
        process(e);
}
```

a felsorolható objektum

Objektumelvű felsorolás

- A felsorolás akkor felel meg az egyszeres felelősség elvének, ha a gyűjtemény nem önmaga felsorolója: a felsorolásának műveletei egy olyan objektum metódusai, amely nyilvántartja a felsorolás állapotát.

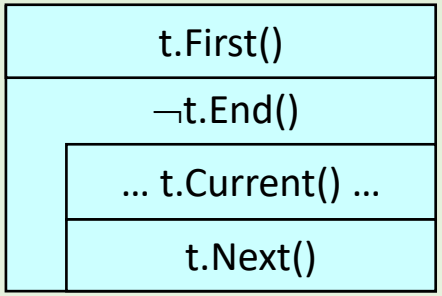


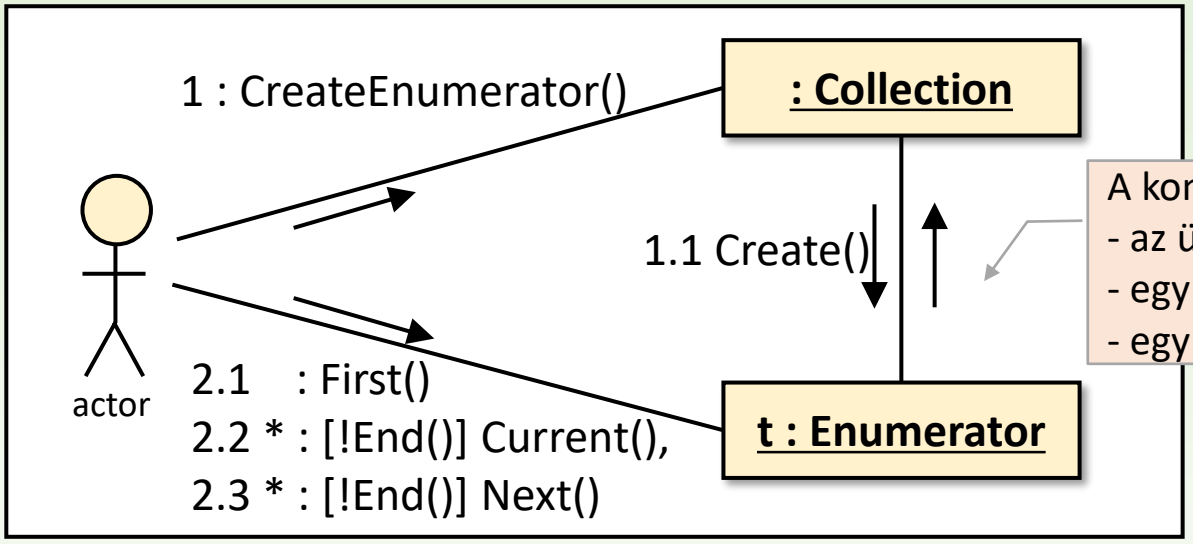
eset		leírás
Előkészítés normális esetben	GIVEN	a felsorolni kívánt gyűjtemény
	WHEN	felsoroló példányosítása
	THEN	létrejön a felsoroló objektum
Előkészítés abnormális esetben	GIVEN	nincs felsorolni kívánt gyűjtemény
	WHEN	felsoroló példányosítása
	THEN	hiba, felsoroló objektum nem jön létre
Elindítás normális esetben	GIVEN	felsoroló állapota pre-start
	WHEN	felsorolás elindítása a first() művelettel
	THEN	a felsoroló in-process állapotba kerül
Elindítás abnormális esetben	GIVEN	felsoroló állapota in-process vagy finished
	WHEN	Felsorolás elindítása a first() művelettel.
	THEN	hiba, és a felsoroló megőrzi állapotát
...		

Felsoroló objektum

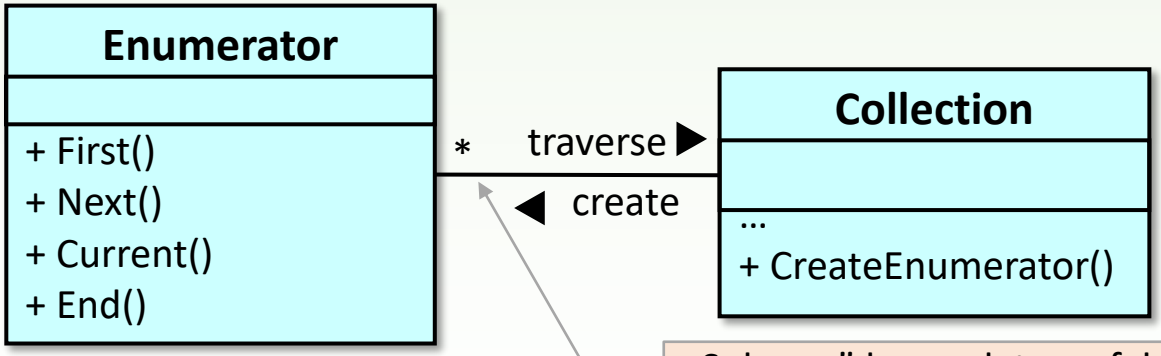
- Azt az objektumot nevezzük felsoroló objektumnak (t : enor(E)), amely rendelkezik egy adott gyűjtemény felsorolásához szükséges First(), Next(), End(), Current() műveletekkel.
- A felsoroló objektum reprezentációja tartalmazza
  - a felsorolni kívánt gyűjtemény hivatkozását
  - a műveletek implementációjához szükséges segéd adatokat.
- Ha a felsoroló objektum elkülönül a felsorolandó gyűjteménytől, akkor
  - egy gyűjteményen egyszerre több felsoroló is dolgozhat
  - a felsorolót a felsorolni kívánt gyűjtemény hozza létre (egyik metódusával), és így a gyűjtemény értesül arról, hogy őt felsorolják.

Kommunikációs diagram





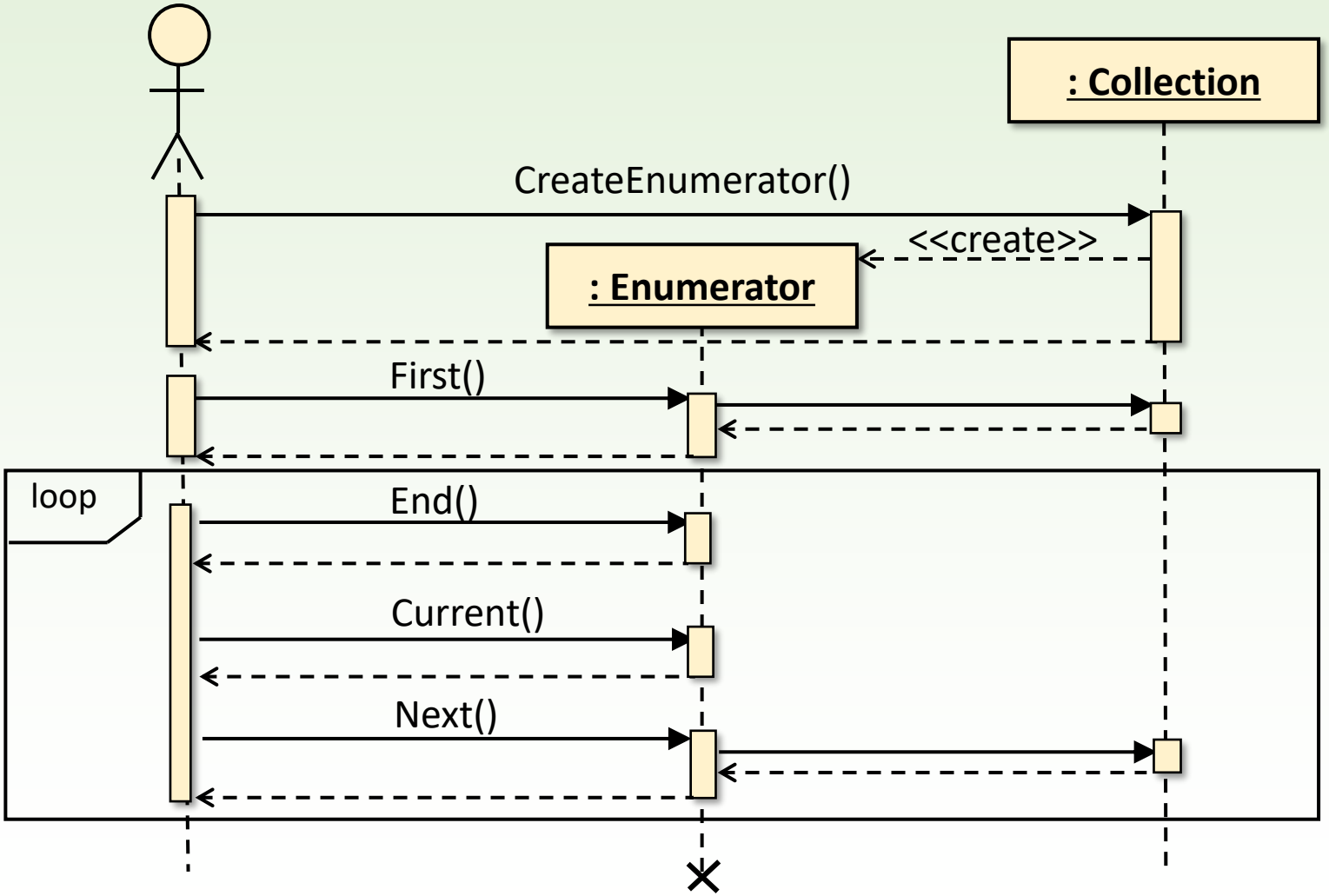
A kommunikációs diagram jelölheti:
- az üzenetek közötti sorrendet
- egy üzenet ismételt hívásait
- egy üzenet elküldésének feltételét



„Sok-egy” kapcsolat van felsorolók és gyűjtemények között: egy gyűjteménynek nulla, egy, vagy több felsorolója is lehet, egy felsoroló pontosan egy gyűjteményhez tartozik.

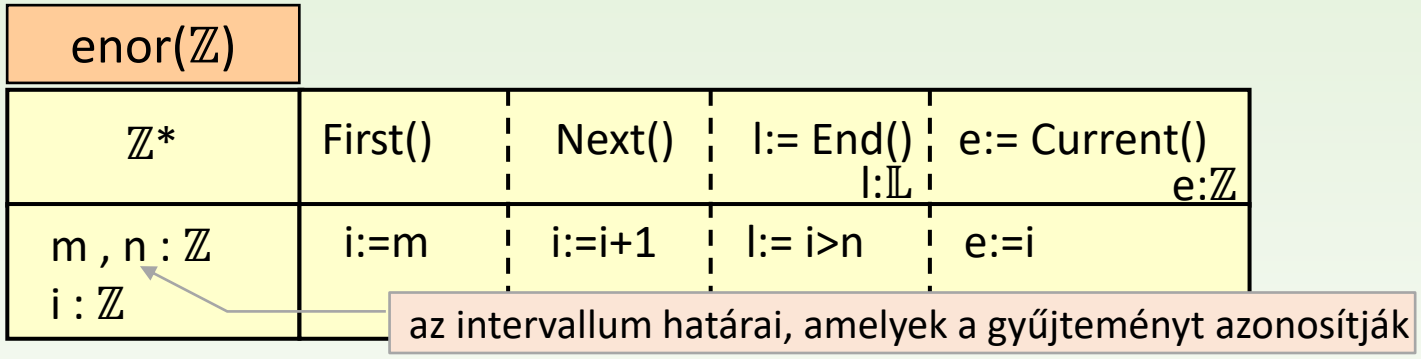
Szekvencia diagram

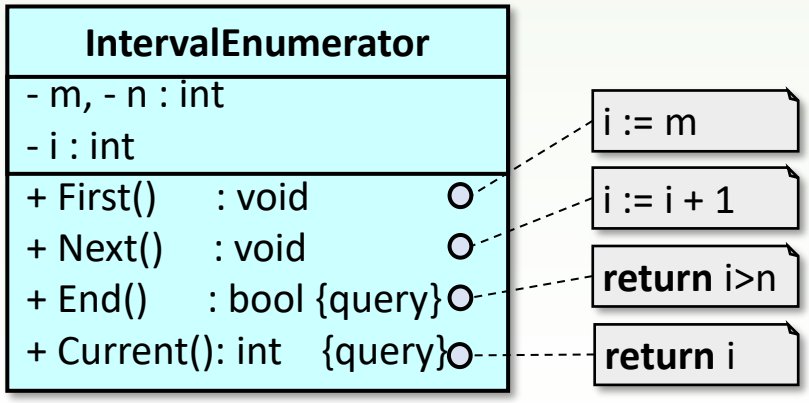
t.First()
¬t.End()
... t.Current() ...
t.Next()

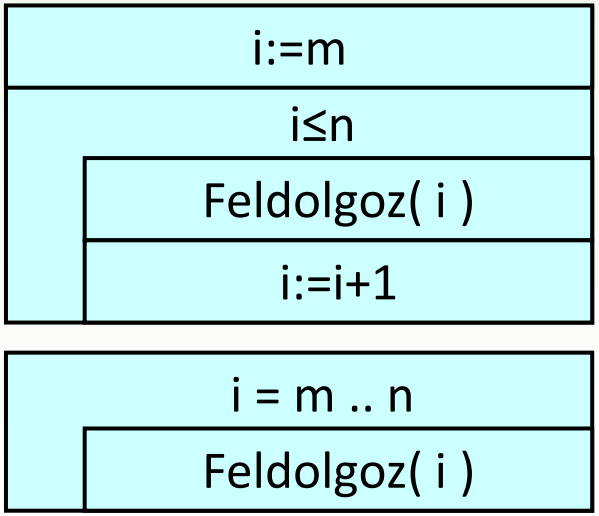


Intervallum klasszikus felsorolója

Egész számok intervallumába eső egész számok felsorolása elejétől a végéig.

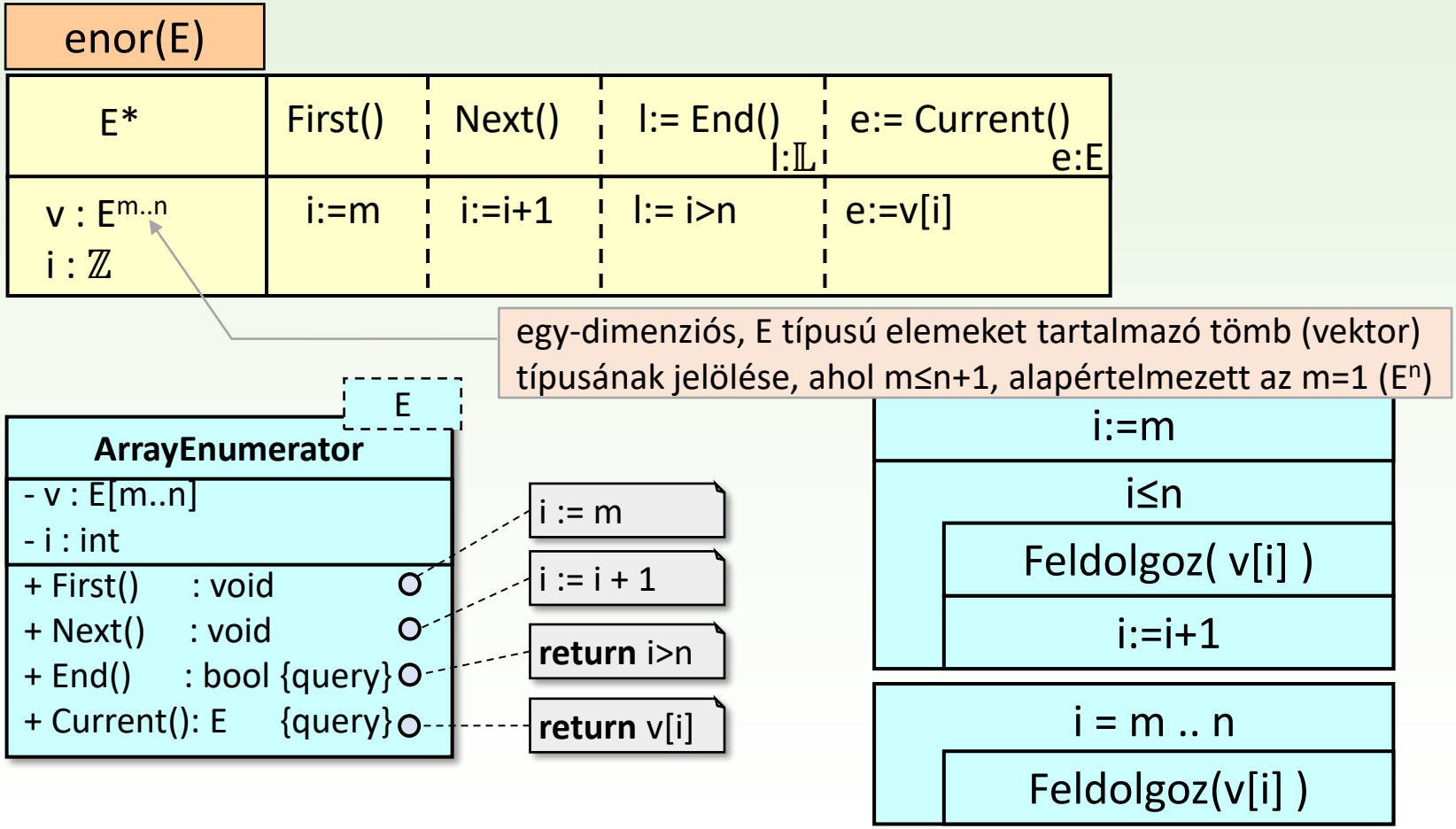






Vektor klasszikus felsorolója

E-beli értékekből álló egy-dimenziós tömb elemeinek felsorolása elejétől a végéig



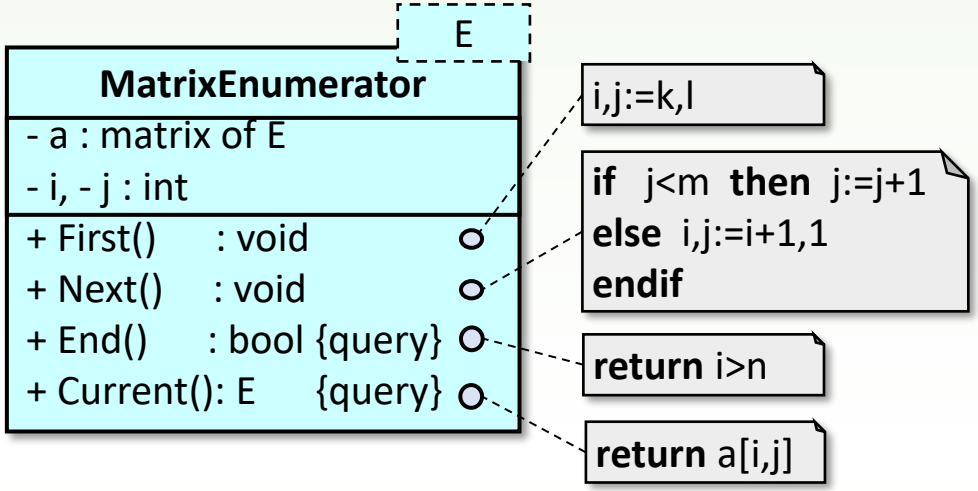
Mátrix sorfolytonos felsorolója

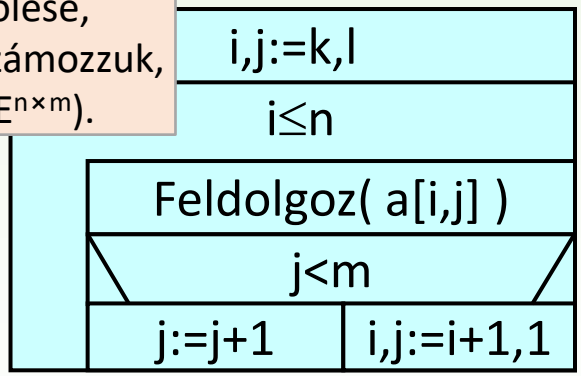
E-beli értékekből álló mátrix elemeinek felsorolása sorfolytonos sorrendben

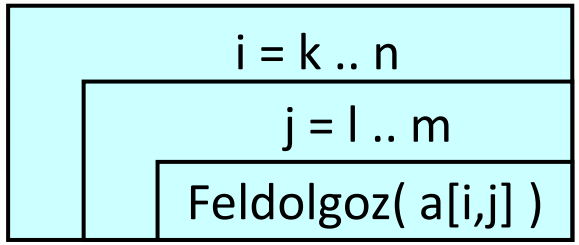
enor(E)

E*	First()	Next()	l:= End() l:𝕃	e:= Current() e:E
a : $E^{k..n \times l..m}$ i,j : ℤ	i,j:=k,l	**if** j<m **then** j:=j+1 **else** i,j:=i+1,l	l:= i>n	e:= a[i,j]

E típusú elemeket tartalmazó mátrix típusának jelölése, ahol a sorokat k-tól n-ig, az oszlopokat l-től m-ig számozzuk, és k≤n+1, l≤m+1, alapértelmezett a k=1 és az l=1 ($E^{n \times m}$).







Algoritmus minták általánosítása

- Algoritmus minták vektorra:
  - i az a:$E^{m..n}$ tömb felsorolója
  - felt:E → 𝕃
    Feldolgoz(f(a[i]), felt(a[i]))

```
i:=m
i≤n
    Feldolgoz(a[i])
    i:=i+1
```

- Algoritmus minták intervallumon értelmezett függvényre:
  - i az [m .. n] intervallum felsorolója
  - f : [m .. n] → H, felt : [m .. n] → 𝕃
    Feldolgoz(f(i), felt(i))

```
i:=m
i≤n
    Feldolgoz(i)
    i:=i+1
```

- Algoritmus minták felsorolóra:
  - t : enor(E) felsoroló
  - f : E → H, felt : E → 𝕃
    Feldolgoz(f(t.Current()), felt(t.Current()))

```
t.First()
¬t.End()
    Feldolgoz( t.Current() )
    t.Next()
```

```
e ∈ t
    Feldolgoz( e )
```

Összegzés

Összegezzük egy felsorolás elemeihez rendelt értékeket!

$f : E \to H$

$+ : H \times H \to H$

$0 \in H$ *baloldali neutrális elem*

$A = (t{:}enor(E), s{:}H)$

$Ef = (t = t')$

$Uf = (s = \Sigma_{e \in t'} f(e))$

A felsorolás végén t = <> , azaz nem marad t = t'

$s = \Sigma_{i=1..|t'|} f(t'_i)$

speciális eset: feltételes összegzés

$$\sum_{\substack{e \in t' \\ felt(e)}} g(e) \text{ azaz } f(e) = \begin{cases} g(e) & \text{ha } felt(e) \\ 0 & \text{különben} \end{cases}$$

$g : E \to H, felt : E \to \mathbb{L}$

```
s := 0
t.First()
¬t.End()
    s := s + f(t.Current())
    t.Next()
```

Számlálás

Számoljuk meg egy felsorolás adott tulajdonságú elemeit!

felt : E → 𝕃

A = (t:enor(E), c:ℕ)

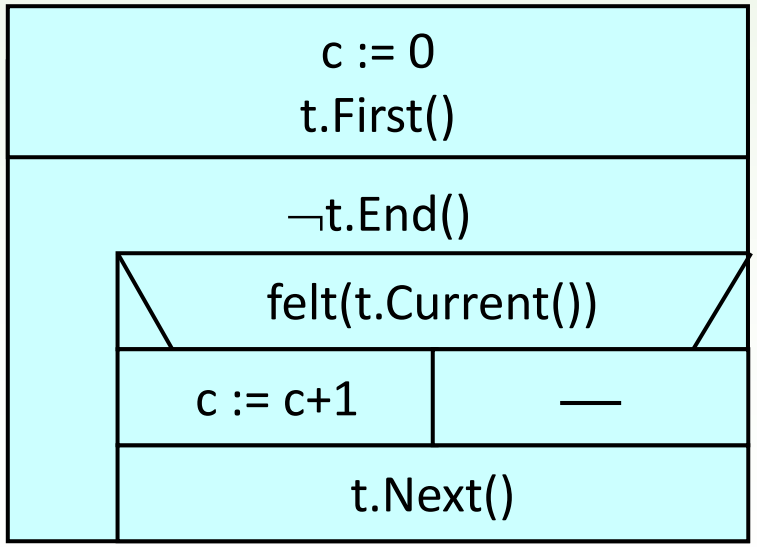
Ef = (t = t')

Uf = (c = $\Sigma_{e \in t'}$ 1)
felt(e)

a természetes számokon értelmezett feltételes összegzés

A számlálás egy speciális összegzés:

$\Sigma_{e \in t'}$ f(e) azaz f(e)= { 1 ha felt(e) ; 0 különben }



Maximum kiválasztás

Adjuk meg egy felsorolás adott szempont szerinti egyik legnagyobb elemét és annak értékét!

$f : E \rightarrow H$

H *halmaz elemei rendezhetőek*

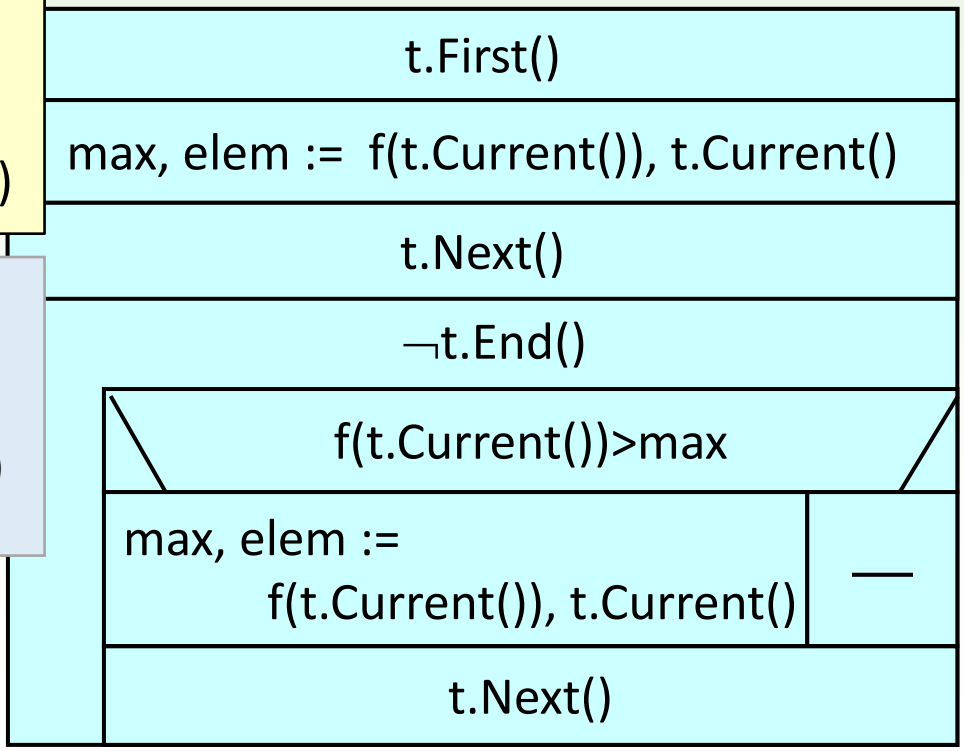
A = (t:enor(E), elem:E, max:H)

Ef = ($t = t' \wedge |t|>0$)

Uf = ($(max, elem) = \mathbf{MAX}_{e \in t'} f(e)$)

$max = \mathbf{MAX}_{i=1..|t'|} f(t'_i)$
$\wedge$ $elem \in t' \wedge max = f(elem)$
vagy
$ind \in [1..|t'|] \wedge \forall i \in [1..|t'|] : f(t'_i) \leq f(t'_{ind})$
$\wedge$ $elem = t'_{ind} \wedge max = f(elem)$

- MAX *helyett lehet* MIN
- elem *elhagyható,* max *nem*



Kiválasztás (biztosan talál)

Keressük meg egy felsorolás adott tulajdonságú első elemét, ha tudjuk, hogy van ilyen!

felt : E → 𝕃

A = (t:enor(E), elem:E)

Ef = (t = t' ∧ ∃e∈t : felt(e))

Uf = ((elem, t) = **SELECT**$_{e \in t'}$ felt(e))

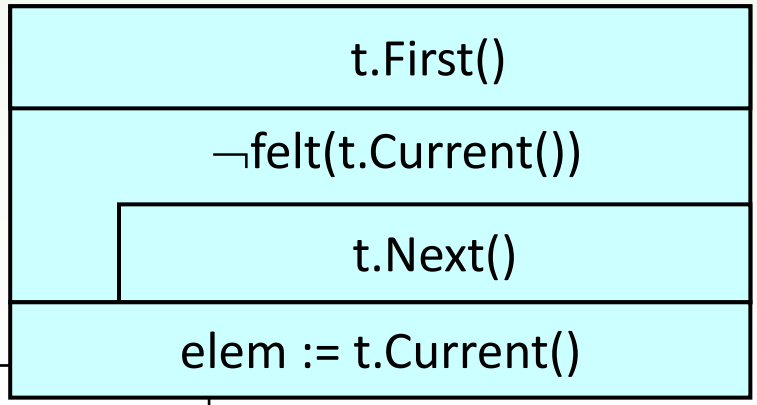
Megadja a *t'* felsorolás első olyan elemét, amelyre a feltétel teljesül (ez lesz az *elem* értéke). A keresés végén a *t* felsoroló *„folyamatban van”* státuszú lesz.

$i \geq 1 \wedge felt(t'_i) \wedge \forall k \in [1..i-1] : \neg felt(t'_k) \wedge elem = t'_i \wedge t = \langle t'_i, \ldots, t'_{|t'|} \rangle$

```
for( t.first(); true; t.next() )
{
    if( felt(t.current()) )
    {
        elem=t.current();  break;
    }
}
```

```
for( auto e : h )
{
    if( felt(e) ) { elem=e;  break; }
}
```

a felsorolható objektum



Lineáris keresés (találat nem biztos)

Keressük meg egy felsorolás adott tulajdonságú első elemét!

felt : E → 𝕃

A = (t:enor(E), l:𝕃, elem:E)

Ef = (t = t')

Uf = ((l, elem, t) = **SEARCH**$_{e \in t'}$ felt(e))

Ha van a *t'* felsorolásnak olyan eleme, amelyre teljesül-e a feltétel, akkor *l* igaz, az *elem* a legelső feltételt kielégítő elem lesz, és lehet, hogy *t „folyamatban van"* státuszban marad. Egyébként *l* hamis, *elem* nem definiált, és *t „befejeződött"* státuszú lesz.

$(l = \forall i \in [1..|t'|] : felt(t'_i)) \wedge$
$(l \longrightarrow i \in [1..|t'|] \wedge felt(t'_i) \wedge \forall k \in [1..i-1] : \neg felt(t'_k)$
$\wedge elem = t'_i \wedge t = <t'_{i+1}, \ldots, t'_{|t'|}>)$

speciálisan eldöntéshez is használhatjuk:
l = **SEARCH**$_{e \in t'}$ felt(e) vagy $l = \exists_{e \in t'}$ felt(e)

l := hamis; t.First()

¬l ∧ ¬t.End()

elem := t.Current()

l := felt(elem)

t.Next()

```
bool l = false;
for( t.first(); !t.end(); t.next() ) {
    if( l=felt(t.current()) ) {
        elem=t.current(); break;
    }
}
```

```
bool l = false;
for( auto e : h ) {
    if( felt(e) ) { elem=e; break; }
}
```

a felsorolható objektum

Optimista lineáris keresés

Vizsgáljuk meg, hogy egy felsorolás minden elemére igaz az adott tulajdonság.
Ha nem, adjuk meg az első olyan elemet, amelyikre nem teljesül!

felt : E → 𝕃

A = (t:enor(E), l:𝕃, elem:E)

Ef = (t = t')

Uf = ((l, elem, t) = ∀**SEARCH**$_{e∈t'}$ felt(e))

Ha a *t'* felsorolás minden elemére teljesül a feltétel, akkor *l* igaz, *elem* nem definiált, és *t „befejeződött"* státuszú lesz. Egyébként *l* hamis, az *elem* a legelső feltételt ki nem elégítő elem lesz, és lehet, hogy a *t* státusza *„folyamatban van"* marad.

(l = ∃i∈[1.. |t'|] : felt(t'$_i$)) ∧
(¬l ⟶ i∈[1.. |t'|] ∧ ¬felt(t'$_i$) ∧ ∀k∈[1..i-1] : felt(t'$_k$)
∧ elem = t'$_i$ és t = <t'$_{i+1}$, … , t'$_{|t'|}$>)

speciálisan eldöntésre is használhatjuk:
l = ∀**SEARCH**$_{e∈t'}$ felt(e) vagy l = ∀$_{e∈t'}$ felt(e)

```
bool l = false;
for( auto e : h ) {
    if( felt(e) ); else { elem=e;  break; }
}
```

a felsorolható objektum

l := igaz; t.First()

l ∧ ¬t.End()

elem := t.Current()

l := felt(elem)

t.Next()

Feltételes maximum keresés

Keressük egy felsorolás adott tulajdonságú elemei között egy adott szempont szerinti egyik legnagyobbat és annak értékét!

$f : E \to H$
$felt : E \to \mathbb{L}$
H *halmaz elemei rendezhetőek*

$A = (t{:}enor(E), l{:}\mathbb{L}, elem{:}E, max{:}H)$

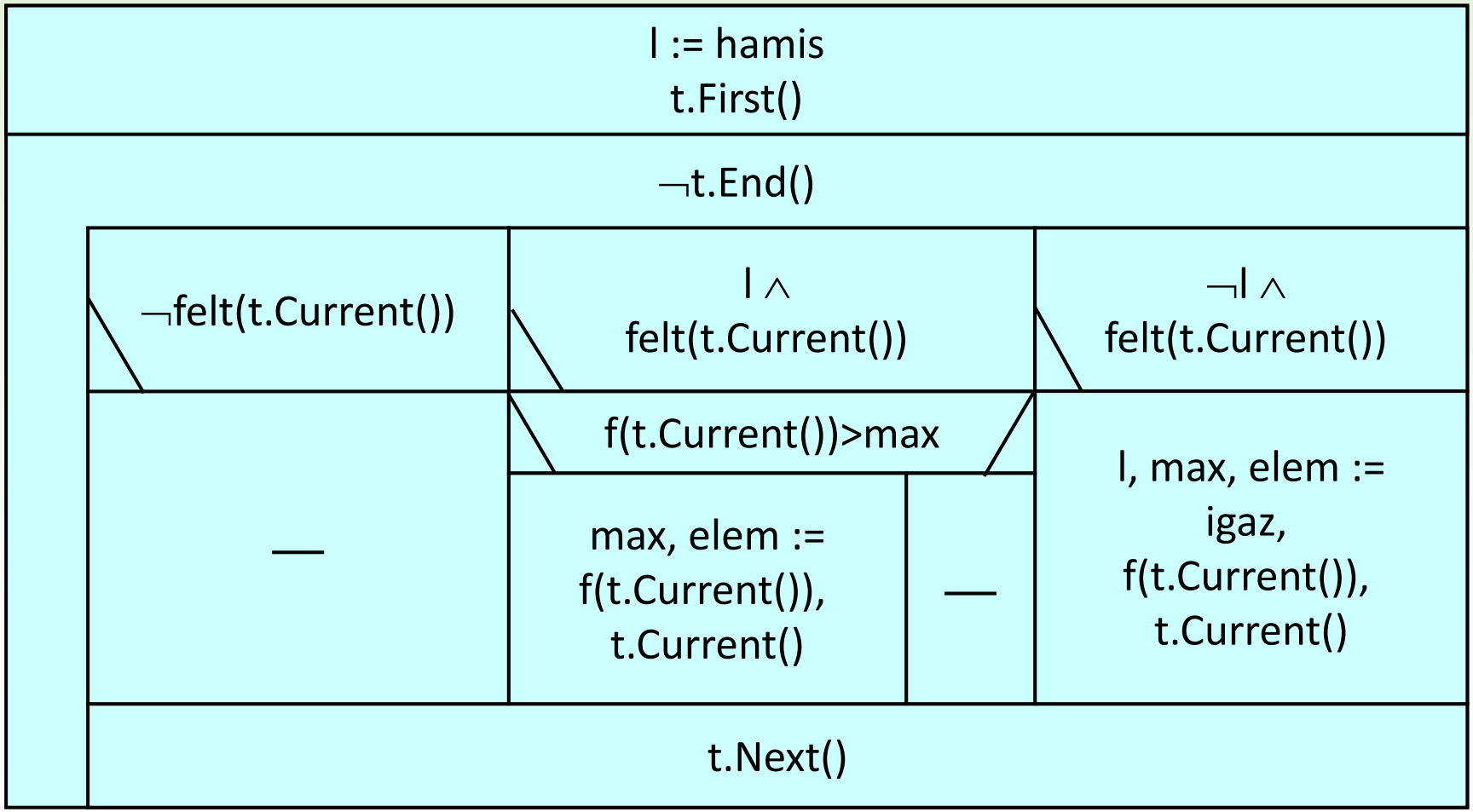
$Ef = (t = t')$

$Uf = ((l, max, elem) = \mathbf{MAX}_{e \in t'}^{felt(e)} f(e))$

Ha van a *t'* felsorolásnak olyan eleme, amelyre teljesül-e a feltétel, akkor *l* igaz, az *elem* a *t'* olyan feltételt kielégítő eleme, amelynek *f* értéke *max*, ami nagyobb vagy egyenlő a *t'* bármelyik olyan elemének *f* értékénél, amely kielégíti a feltételt. Egyébként az *l* hamis, az *elem* és *max* nem definiált.

$$
\begin{aligned}
&(l = \exists i \in [1..|t'|] : felt(t'_i)) \wedge \\
&(l \longrightarrow i \in [1..|t'|] \wedge felt(t'_i) \wedge \\
&\quad \wedge \forall k \in [1..t'|] : (felt(t'_k) \longrightarrow f(t'_k) \leq f(t'_i)) \\
&\quad \wedge elem = t'_i \wedge max = f(elem))
\end{aligned}
$$

Feltételes maximum keresés



- MAX *helyett lehet* MIN
- elem *elhagyható,* max *nem*

Visszavezetés

1. Megsejtjük a feladatot (részfeladatot) megoldó algoritmus mintát.
2. Specifikáljuk a feladatot az algoritmus mintára utaló utófeltétellel.
3. Rögzítjük a feladat és az algoritmus minta közötti eltéréseket:
   - a *felsoroló* és a *felsorolt elemek* típusát
   - a *függvények* (f : E→H, felt : E→$\mathbb{L}$) konkrét megfelelőit
   - a H halmaz *műveletét*, ha kell
     - (H, >) helyett például ($\mathbb{Z}$, >) vagy ($\mathbb{Z}$, <)
     - (H, +) helyett például ($\mathbb{Z}$, +) vagy ($\mathbb{R}$, *) vagy ($\mathbb{L}$, $\wedge$)
   - a *változók átnevezéseit*
4. Beleírjuk az algoritmus minta algoritmusába a fenti különbségeket, és így megkapjuk a feladatot megoldó algoritmust.

Programozási tétel:
Ha egy feladat és egy algoritmus minta alapfeladata megfeleltethetők egymásnak, akkor az algoritmus minta algoritmusának ezen megfeleltetés szerint átalakított változata megoldja a kitűzött feladatot.

Tesztelési stratégiák

- Fekete doboz: a feladat (specifikációja) alapján felírt tesztesetek.
  - az előfeltételt megszegő ún. érvénytelen tesztesetek
  - az utófeltétel eseteinek vizsgálata
  - …

- Fehér doboz: a kód alapján felírt tesztesetek.
  - algoritmus minden utasításának kipróbálása
  - algoritmus minden vezérlési csomópontjának (elágazás, ciklus) kipróbálása
  - …

- Szürke doboz: végrehajtható specifikáció által előrevetített algoritmus működését ellenőrző tesztesetek.
  - Ha ez a végrehajtható specifikáció algoritmus mintákra utal, akkor az algoritmus minták szokásos teszteseteit kell vizsgálni.

Algoritmus minták tesztesetei

- Felsoroló szerint (mindegyik algoritmus minta esetén)
  - eltérő *hosszúságú* felsorolások: nulla, egy illetve hosszabb felsorolásokra is kipróbájuk az algoritmust
  - Felsorolás kezdete és vége: feldolgozza-e az algoritmus a felsorolás *első* ill. *utolsó* elemét
- Funkció szerint (algoritmus mintánként eltérő)
  - *összegzés*: felsorolás hosszának skálázása
  - *keresés*, *számlálás*: van vagy nincs keresett tulajdonságú elem
  - *max. kiv.*: egyetlen, illetve több azonos maximális érték
  - *felt. max. ker.*: - van vagy nincs keresett tulajdonságú elem
    - feltételt kielégítő egyetlen, illetve több azonos maximális érték
    - a legnagyobb értékű elem nem elégíti ki a feltételt
- A *felt*(*e*) és *f*(*e*) kifejezések kiszámolásánál használt műveletek sajátosságai.

Feladat

A Föld felszín egy vonalán adott pontokon megmértük a felszín tengerszint feletti magasságát, és az adatokat egy tömbben tároltuk el. Hol található és milyen magas a felszín legmagasabb horpadása?

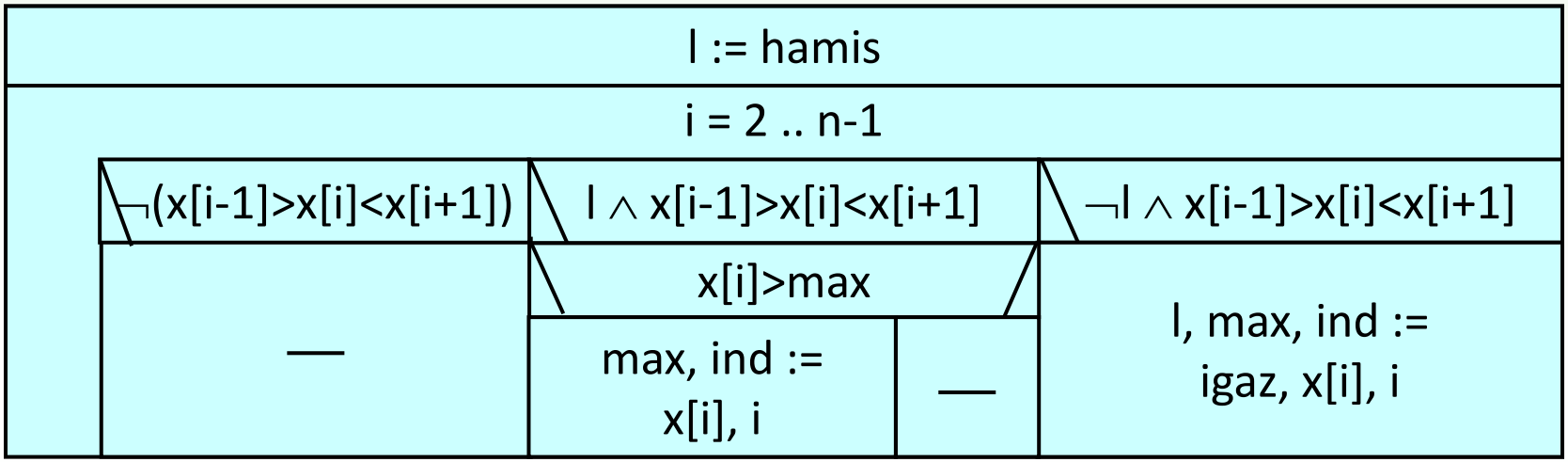
$A = (x : \mathbb{R}^n , l : \mathbb{L}, max : \mathbb{R}, ind : \mathbb{N})$

$Ef = (x = x_0)$

$Uf = (Ef \wedge (l, max, ind) = \mathbf{MAX}_{i=2..n-1,\ x[i-1]>x[i]<x[i+1]} x[i])$

Feltételes maximumkeresés:

e∈t:enor(E)	~	i∈[2 .. n-1]
f(e)	~	x[i]
felt(e)	~	x[i-1]>x[i]<x[i+1]
H, >	~	ℝ, >



Főprogram

```
vector<double> FillInFromFile(const string &str);
bool maxsearch(const vector<double> &x, double& max, unsigned int& ind);

int main()
{
	vector<double> x = FillInFromFile("input.txt");

	double max=0.0;
	unsigned int ind=0;
	if(maxsearch(x, max, ind))
		cout << " The height of the highest depression: " << max
			<< " on the place " << ind << endl;
	} else {
		cout << "There is no depression." << endl;
	}
	return 0;
}
```

Feltételes maximumkeresés kódja

```
bool maxsearch(const vector<double> &x, double& max, unsigned int& ind)
{
    bool l = false;
    for(unsigned int i=1; i<x.size(); ++i) {
        if (!(x[i-1]>x[i] && x[i]<x[i+1])) continue;
        if (l) {
            if ( max<x[i] ) {
                max = x[i]; ind = i;
            }
        } else {
            l = true; max = x[i]; ind = i;
        }
    }
    return l;
}
```

ciklusmag végére ugrik a vezérlés

Tesztelés

Feltételes maximum keresés tesztesetei			
felsoroló szerint	hossza: 0	x = < > , < 1.0, 2.0>	→ l = hamis
	hossza: 1	x = < 2.0, 1.0, 2.0 >	→ l = igaz, max = 1.0, ind = 2
	hossza: több	x = < 1.0, 2.0, 1.0, 4.0, 2.0 >	→ l = igaz, max = 1.0, ind = 3
	eleje	x = < 3.0, 2.0, 3.0, 1.0, 3.0 >	→ l = igaz, max = 2.0, ind = 2
	vége	x = < 3.0, 1.0, 3.0, 2.0, 3.0 >	→ l = igaz, max = 2.0, ind = 4
tétel szerint	nincs	x = < 1.0, 2.0, 3.0, 4.0, 5.0 >	→ l = hamis
	van	x = < 1.0, 2.0, 1.0, 4.0, 2.0 >	→ l = igaz, max = 1.0, ind = 3
	egy maximum	x = < 3.0, 1.0, 3.0, 2.0, 3.0 >	→ l = igaz, max = 2.0, ind = 4
	több maximum	x = < 3.0, 2.0, 3.0, 2.0, 3.0 >	→ l = igaz, max = 2.0, ind = 2